Docker

23 March 2023 02:05 PM

1. Virtualization (The Concept)

Virtualization is the process of taking one physical computer (a bare-metal server) and tricking it into acting like multiple, completely separate virtual computers.
Instead of buying 10 physical servers that each sit 80% idle, virtualization allows you to run 10 "Virtual Machines" (VMs) on a single physical machine, sharing its CPU, RAM, and storage efficiently.

- **Without Virtualization:** 1 physical server = 1 operating system (OS).
- **With Virtualization:** 1 physical server = Many isolated VMs, each with its own OS (Debian, Ubuntu, Windows, etc.).

2. Hypervisor (The Software/Manager)

If Virtualization is the concept, the **Hypervisor** is the software manager that actually makes it happen.
The hypervisor sits on top of the physical hardware, slices up the real CPU cores, RAM, and disk space, and hands out "virtual" portions to each VM. It acts like a **traffic cop** to ensure VM #1 cannot spy on, crash, or steal resources from VM #2.
There are two types of Hypervisors:

1. **Type 1 (Bare-Metal):** Runs directly on the physical hardware (e.g., KVM, VMware ESXi). Fast, enterprise-grade, highly secure.
2. **Type 2 (Hosted):** Runs on top of an existing OS like a regular app (e.g., VirtualBox, VMware Workstation). Mostly used for local testing on personal laptops.

🔄 Dual-Booting vs. Virtualization

- **Dual-Boot (Bare-Metal):** Shares hard drive space, but you can only boot into **one OS at a time**. To switch, you must fully reboot the machine. Idle OS environments waste 100% of the hardware.
- **Virtualization (Hypervisor):** Runs **multiple OSs side-by-side simultaneously**. Switching is instant (like clicking a window). Resources are shared dynamically in real time, maximizing efficiency.

Think of a physical server as an **apartment building**, and your compute workloads as the **tenants**.

☁️ Cloud Perspective (Google Cloud / GCP)

- **Your View:** When you spin up a GCP Compute Engine instance, Google hands you **one VM with one OS**.
- **Google's View:** Google's massive physical datacenter hardware runs a **Type-1 Hypervisor** that hosts your Linux VM, another customer's Windows VM, and a third customer's Red Hat VM **all at the exact same time on the same machine**.

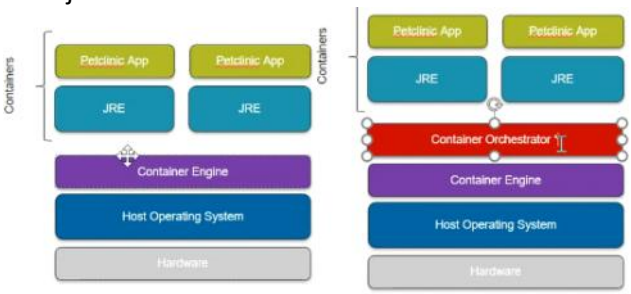
💥 Why Docker and VirtualBox Clash on a PC

1. Your physical CPU has a single virtualization driver channel (Intel VT-x or AMD-V). Only **one** virtualization engine can grab the wheel at a time.
2. When you install **Docker Desktop** on Windows, it enables native **Hyper-V** or **WSL2**. This runs as a Type-1 Hypervisor and locks the CPU virtualization channels immediately upon boot.

3. When you open **VirtualBox** (Type-2), it tries to access those same CPU channels, finds them completely locked by Hyper-V, and fails to launch or throw errors (VT-x is not available).

Container Engine: *It maintain the containers, a container is a small package which has /usr/bin and /usr/lib files.*

If you think of an executable (in /usr/bin) as a "worker," then /usr/lib is the "toolbox" they share to get their jobs done.



Application Deployment Comparison Matrix

Feature / Criteria	1. Physical Server (Bare Metal)	2. Virtual Machine (VM)	3. Containers (Docker, etc.)	4. Container Orchestrator (Kubernetes)
Resource Optimization	❌ **Poor / Not Optimized** *(Hardware is heavily underutilized)*	⚠ **Good** *(Better hardware sharing via Hypervisor)*	⚠ **Good** *(Lightweight, shares Host OS kernel)*	**Excellent** *(Dynamically allocates resources based on real-time load)*
Runtime Environment	🔒 **Restricted** *(Single runtime environment per server)*	🔒 **Restricted** *(Single runtime environment per VM)*	**No Restrictions** *(Each container holds its own isolated runtime)*	**No Restrictions** *(Isolated runtimes across standard container footprints)*
Scalability Costs	💰💰💰💰 **Very High** *(Requires buying & provisioning new physical hardware)*	💰💰💰 **High** *(Spinning up full VMs takes time and overhead)*	💰💰💰 **High** *(Manual scaling of raw containers becomes expensive/complex)*	💰 **Lower Costs** *(Automated, instant horizontal scaling up and down)*
Operational Costs	💸💸💸💸 **Very High** *(Heavy maintenance, cooling, space, manual provisioning)*	💸💸💸💸 **Very High** *(OS licensing, patching, and hypervisor management overhead)*	💸💸 **High** *(High manual effort required to manage networking/storage at scale)*	💸 **Lower Costs** *(Self-healing, automated rollouts, and declarative configurations)*
Resource / Host Limits	🔘 **Highly Limited** *(Bound strictly to a single box)*	🔘 **Limited** *(Constrained by the physical host machine's total resources)*	**Virtually Unlimited** *(Pack more containers per host, but still bound to host limit)*	**Virtually Unlimited** *(Abstracts multiple hosts into a single giant cluster pool)*
Key Architectur	Direct Hardware	**Hypervisor** *(Software that*	**Container Engine** *(Isolates apps at*	**Control Plane / Scheduler**

al Component		*creates and runs separate VMs)*	*the process level)*	*(Automating deployment & management across a cluster)*

***Container Orchestrator**: It is like Kubernetes, where it will manage whole containerization.*

Benefits of Containerization:

- Easy to setup and run
- Lower costs than VM
- Fault isolation for application
- Portable across workspaces
- Low operational costs with orchestration platform.

Tools for container engine:

- Docker
- Podman
- Rkt(Rocket)
- RUNC
- Containerd

Restricting what the process sees:

- Limit access to filesystem with chroot
- Limit access to network, mount, inter-process communication with namespaces.

Restricting the resources used by the process:

- Limit the process CPU, memory and storage with cgroups.

Container:

A **Container** is a lightweight, standalone package that contains an application and everything it needs to run (code, runtime, system tools, libraries, and settings).

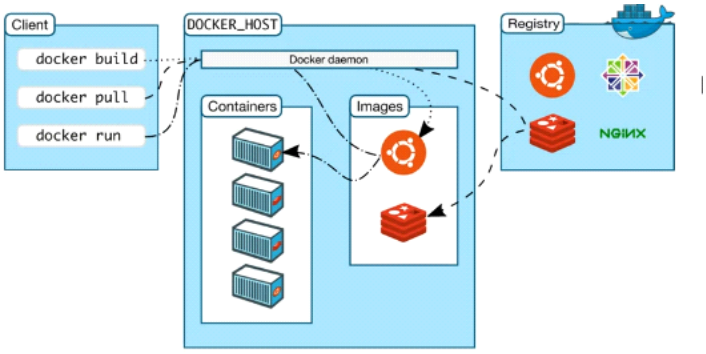
What exactly docker container contains before docker run:

- **System Libraries:** Low-level C libraries (like glibc or musl) that allow code to interact with the host system's hardware kernel.
- **Basic Unix Utilities:** Standard command-line tools like ls, sh, cat, and grep.
- **Security Certificates:** Root SSL certificates so your application can securely connect to HTTPS websites.

DOCKER:

- Docker is an open source containerization platform.
- It enables developers to package applications into containers—standardized executable components combining application source code with the operating system (OS) libraries and dependencies required to run that code in any environment.
- ***"Docker is a containerization platform that allows us to package applications along with dependencies into lightweight, portable containers ensuring consistency across environments."***

Docker Architecture:



Client:

- Docker build - It will create a Docker image./
- Docker pull - It pulls the container image from the registry.
- Docker Run - It will run the container with the image we pulled.

Registry: Here we maintain the images, It stores the images what we build, and provide versioning.

Docker Host:

- ***Docker Daemon:*** A persistent background process that manages Docker images, containers, networks, and storage volumes.
- Ex: When you use docker run command to start up a container, your docker client will translate that command into http API call, sends it to docker daemon, Docker daemon then evaluates the request, talks to underlying OS and provisions your container.

Docker Objects:

Dockerfile --(BUILD)-->Docker Image --(RUN)-->Docker Container.

Dockerfile: It is a text document that contains the instructions on building Docker image.

How

An example of using Docker would be a developer who wants to create a web application using the Python programming language and the Flask web framework. The developer creates a Dockerfile that specifies the base image (e.g. python:3.9) to use, along with the instructions to install the necessary dependencies (e.g. Flask, gunicorn, etc.).
The developer then builds the Docker image from the Dockerfile, and runs a container from the image. The container runs on the host machine, but is isolated from the host's system and can only access the resources that have been specifically allocated to it.
The developer can then test the web application inside the container, and once it is working as expected, the developer can push the container image to a registry like Docker Hub, so that it can be easily shared with other developers or deployed to production.

DockerFile extension:

A Dockerfile normally has **no file extension at all**.
By standard convention, the file is simply named Dockerfile (with a capital "D" and no .txt or .sh extension).

Common Naming Patterns

- **Standard / Default:** Dockerfile
- **Multiple Dockerfiles in the same directory:**
  - Dockerfile.dev
  - Dockerfile.prod
  - Dockerfile.test

When using a non-standard name like Dockerfile.dev, explicitly point to it during the build step using the -f flag:

```
docker build -f Dockerfile.dev -t my-app:dev .
```

Docker Architecture (Daemon vs. CLI)

Interviewers want to know if you understand that Docker is a **Client-Server** application.

- **Docker CLI (The Client):** When you type docker build, you are talking to the client.
- **Docker Daemon (dockerd):** This is the "Brain" that actually does the work. It lives on the host and manages images/containers.
- **The Secret:** The Client and Daemon don't have to be on the same machine! They talk over a **REST API**.

Commands:

FROM:	Specifies the base image to use for the Docker image.
COPY:	Copies files and directories from the host machine to the Docker image.
ADD:	Similar to COPY, but also allows downloading files from URLs and extracting tar archives.
RUN:	Executes commands during the build process to install dependencies, configure the environment, etc.
WORKDIR:	Sets the working directory inside the Docker image for subsequent commands.
ENV:	Sets environment variables inside the Docker image.
EXPOSE:	Specifies which ports should be exposed by the Docker container.
CMD:	Specifies the default command to run when the container starts. It can be overridden by passing a command to the docker run command.
ENTRYPOINT:	Specifies the entry point for the container, which is the command that is executed when the container starts. It can be used in combination with CMD to provide default arguments.
VOLUME:	Creates a mount point for a volume or a directory from the host machine to the Docker container.
USER:	Sets the user or UID that the Docker container should run as.
LABEL:	Adds metadata to the Docker image in the form of key-value pairs.

```
# Use an official Node.js runtime as the base image
FROM node:14
```

```
# Set the working directory inside the container
WORKDIR /app

# Copy package.json and package-lock.json to the working directory
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy the rest of the application code to the working directory
COPY . .
```

- **The first . (Source):** This means "Everything in the current folder on my computer" (where the Dockerfile is).
- **The second . (Destination):** This means "Put them in the current working directory inside the container" (which is /app, because you just set that with WORKDIR).
  Note: If you didn't have WORKDIR /app, the second . would copy your files into the **root** directory (/), which makes the container very messy and disorganized.

```
# Expose port 3000 to the outside world
EXPOSE 3000

# Define the default command to run the application
CMD ["node", "app.js"]
```

Difference between CMD and Entrypoint:

1. CMD vs. ENTRYPOINT

- **ENTRYPOINT:** This is the **"Fixed"** command. It makes your container behave like a specific executable.
- **CMD:** This is the **"Default Argument"**. It can be easily overridden by the user at runtime.
  Analogy: If your container is a "Car," the **ENTRYPOINT** is the engine (it always runs). The **CMD** is the destination on the GPS (it's there by default, but you can change it whenever you want).

Interview Answer: *"I use ENTRYPOINT for the main command that should always run, and CMD to provide default arguments that the user might want to change."*

Example:

Most professional Dockerfiles use them together.
Example Dockerfile:
Dockerfile

```
ENTRYPOINT ["python3"]
CMD ["app.py"]
```

- **Case A (Default):** Running docker run my-image executes:
  python3 app.py
- **Case B (Override):** Running docker run my-image test.py executes:

python3 test.py
(Notice how python3 stayed, but app.py was replaced).

Docker Layers:

In Docker, a container image is composed of multiple layers, where each layer represents a set of filesystem changes. These layers are stacked on top of each other to form the final image. Docker uses a copy-on-write filesystem (usually based on UnionFS) to efficiently manage these layers. Here's an overview of Docker layers: (multiple bricks)

Base Image Layer:

- The base image layer is the initial layer of the image and serves as the foundation for subsequent layers.
- It typically contains the root filesystem of the operating system (e.g., Ubuntu, Alpine) or a pre-built application environment (e.g., Python, Node.js).

Intermediate Layers:

- Intermediate layers represent additional filesystem changes or modifications made on top of the base image layer.
- Each Dockerfile instruction (e.g., RUN, COPY, ADD) results in the creation of a new intermediate layer.
- These layers capture changes such as installing packages, copying files, setting environment variables, and running commands specified in the Dockerfile.

Top Layer (Writable Layer):

- The top layer, also known as the writable layer or the container layer, is the final layer of the image.
- It is where any changes made to the container's filesystem at runtime are stored.
- This layer is ephemeral and unique to each container instance, allowing containers to be lightweight and stateless.

Interview Question

"How does the order of commands in a Dockerfile affect **build speed**?"

Docker uses a Layer Cache. You should put the instructions that change least often (like installing OS packages) at the top, and things that change every time (like copying source code) at the bottom.

Analogy:

Each line in your Dockerfile (like FROM, RUN, or COPY) creates a new "brick" (a layer) and places it on top of the previous one.

1. How the "Cache" works

When you build a Docker image for the second time, Docker looks at your instructions and asks: *"Did this line change since last time?"*

- **If NO:** Docker says, "Great! I already have the brick for this," and it reuse the old one instantly. This is the **Cache Hit**.
- **If YES:** Docker says, "I need to build a new brick." This is a **Cache Miss**.

2. The "Domino Effect"

Here is the most important rule of Docker builds: **If one layer changes, every single layer after it must be rebuilt from scratch.** Docker cannot reuse a "top brick" if the "bottom brick" has changed.

EX:

```
FROM node:18
WORKDIR /app

# 1. Copy ONLY the dependency list (This rarely changes)
COPY package.json .

# 2. Install dependencies (Cached unless you add a new library)
RUN npm install

# 3. Copy the rest of the source code (Changes often)
COPY . .
```

Summary Checklist for Speed

Frequency of Change	Examples	Position in Dockerfile
Never/Rarely	FROM, WORKDIR, ENV	**Top**
Infrequently	apt-get install, npm install	**Middle**
Constantly	COPY . . (Source Code)	**Bottom**

Interview Question

How to Reduce layers?

To attain a smaller image by reducing layers, you need to understand one rule: **Each RUN, COPY, and ADD instruction in a Dockerfile creates a new layer.**

1. How to Combine RUN Commands

Use the shell operator **&&** to chain commands together and the backslash **** to break them into multiple lines for readability.

```
RUN apt-get update && apt-get install -y python3 && rm -rf
/var/lib/apt/lists/*
```

2. The "Delete in the Same Layer" Rule

This is the most important concept for image size. Docker layers are **additive**.

- If you download a 100MB file in Layer 1 and delete it in Layer 2, the final image is **still 100MB larger** because Layer 1 is hidden underneath.
- To actually save space, you must **download and delete in the same RUN command.**

Correct Example:

```
RUN wget http://example.com/big-file.tar.gz && \
    tar -xvf big-file.tar.gz && \
    rm big-file.tar.gz
```

Because the rm happens before the RUN command finishes, that 100MB file is never saved to a layer.

How?

1. **wget ...:** Docker downloads the compressed package (big-file.tar.gz). Let's say this archive is **100MB**.
2. **tar -xvf ...:** Docker unzips and extracts the archive. It explodes into a folder containing your actual application files (e.g., code, binaries, assets). Let's say the extracted files take up **250MB**.
3. **rm ...:** Docker completely deletes the original **100MB** compressed .tar.gz file.
   The **250MB** of extracted files are left completely untouched!

Image Optimization (IMPORTANT)

Techniques:

☑ Use lightweight base image

alpine instead of ubuntu (apline is light weight linux)

☑ Reduce layers

Combine RUN commands

☑ Remove cache

```
RUN apt-get update && apt-get install -y python3 && apt-get clean &&
rm -rf /var/lib/apt/lists/*
```

To actually save space, **you must run apt-get clean (and other cleanup commands) in the exact same RUN instruction (Same layer) where you installed the software.**
If you do it in a separate RUN line later, it's too late—the "heavy" files are already baked into the previous layer forever.

The "Wrong" Way (Two Layers)

Even though it looks like you are cleaning up, you aren't saving any space.
Dockerfile

RUN apt-get update && apt-get install -y python3
RUN apt-get clean && rm -rf /var/lib/apt/lists/*

- **Result:** * **Layer 1:** Contains python3 + 50MB of cache/installers.
  - **Layer 2:** Deletes the 50MB of files.
  - **Total:** The image still weighs the same because Layer 1 is still sitting underneath Layer 2.

Analogy:

The "Transparency" Analogy

Imagine you have three clear plastic sheets:

- **Sheet 1 (Base):** You draw a circle.
- **Sheet 2 (The File):** You draw a **big red square**.
- **Sheet 3 (The Cleanup):** You take a piece of **white tape** and stick it over the red square.

When you look at the stack from the top, **the red square is gone.** You only see the circle. However, if you weigh those three sheets, **the weight of the red square is still there** because the ink is still on Sheet 2!

In Docker, a "Layer" is one of those plastic sheets. Once a layer is created, it is **Read-Only**. It can never be changed or shrunk.

Interview Question

If an interviewer asks: *"Can I reduce the size of an existing heavy image by running a cleanup command in a new container?"*

- **The Answer is NO.** You can't "fix" a heavy image by adding more layers. You have to go back to the **Dockerfile** and fix the RUN command so the junk is never saved in the first place.

Use .dockerignore

the files in .dockerignore never even copies from your local VM to the Docker Engine.

- ## Pro-Tip: The "Inverted" .dockerignore

  Sometimes, it's easier to ignore **everything** and then only "un-ignore" what you actually need. You do this using the * and ! symbols:

```
# Ignore everything
*
# But allow these specific things
!src/
!package.json
!app.py
```

Docker Image: Docker image is a read-only template that contains a set of instructions for creating a container that can run on the docker platform :

- It is a light weight, standalone ad executable package of software that includes everything to run an application.

Image :

- Runtime environments
- Runtime laibraries and dependencies
- Environment variables
- Config. Files
- App code or bin files

Docker Container: They are running instances of Docker image. It shares the kernel with other containers and runs as an isolated process in user space on the host OS.

Container : A container is a lightweight, stand-alone, executable package that includes

everything needed to run a piece of software, including the code, a runtime, libraries, environment variables, and config files.

Commands:

- docker run - Start the container
  - Ex: docker run -I -t --name "HelloWorld" ubuntu:latest /bin/bash
  - docker run -d -p 80:80 nginx
    - -d → detached
    - -p → port mapping

- docker stop - terminate or stop the container
- docker exec - executes the command inside the container.
- docker ps [Options] - get list of running containers

Docker CLI:

- docker build -t <tag> <path>
  - `docker build -t universal-app .`
  - "." indicates - Look in my **current folder** for the Dockerfile and all the files I want to include in my image."
- docker run [options] image [COMMAND] [ARGS…]
- docker tag [OPTIONS] <SOURCE_IMAGE> <DEST_IMAGE>
- docker push/pull [OPTIONS] NAME[:TAG|@DIGEST]
- docker images [OPTIONS] {RESPOSITORY[:TAG]]
- Docker start [OPTIONS] Container
- Docker stop [OPTIONS] Container
- Docker pause Container
- Docker restart [OPTIONS] Container
- Docker exec [OPTIONS] Container Command [ARGS…]
  - Ex: `docker exec -it <container> /bin/bash`

Importance of Docker Tag:

In the context of container management, Docker tagging is a critical process for organizing and deploying images. Here is the breakdown as requested:

What is Docker Tag?

A tag is essentially a label or version identifier assigned to a specific image. It allows you to create an alias for an existing image, typically in the format: `repository-name:tag-name` (e.g., `nginx:latest` or `username/my-app:v1`).

- It won't push docker image, it will simply make docker image name ready to use for push.

Why is it useful?

- Registry Routing: It tells *Docker* exactly which registry and repository to push an image to. Without a tag containing your account username, *Docker* would not know where your private repository is located.
- Versioning: It allows you to keep track of different iterations of your application (e.g., v1, v2, stable) so you can deploy specific versions reliably.

How it is used in the video

In this demonstration (20:42), the presenter uses the tag command to prepare a local image for a private registry:

1. Renaming for Registry: The presenter runs `docker tag nginx:1.14.2 abishekfi/nginx-image-demo:v1` (20:42). (For image to google artifact registry: `docker tag nginx:latest LOCATION-docker.pkg.dev/PROJECT-ID/REPOSITORY/my-nginx:v1`)
2. Destination Binding: By adding the username abishekfi, the image is now "bound" to the specific private repository the user created on *Docker Hub*.
3. Deployment Preparation: Once tagged and pushed, this specific name is referenced in the *Kubernetes* deployment YAML file (23:00), allowing the cluster to attempt to pull that exact private image.

Troubleshooting commands:

1. **docker ps** - list of containers
2. **docker logs** - If a container stops immediately after starting, the "why" is usually in the logs
   a. docker logs <container>
3. **docker inspect** - This gives you every technical detail about the container in JSON format, including IP addresses, mount points, and environment variables.
4. **docker stats** - Is the container frozen because it ran out of RAM? (A live stream of CPU, Memory, and Network usage for all running containers.)
5. **docker events** -
   This shows you a real-time stream of what the Docker engine is doing (starting, killing, or ooming a container).

Command	Best Used For...
docker ps -a	Seeing all containers (even the ones that crashed/exited).
docker top <id>	Seeing which processes are running *inside* the container.
docker port <id>	Verifying which host ports are actually mapped to the container.
docker diff <id>	Seeing what files were changed/added compared to the original image.
docker network ls	Checking if your containers are even on the same network.

Interview Scenario

Container not working

Steps:

1. Check logs
2. Check ports
3. Check env vars
4. Exec inside container

Interview Question

👉 *How do you persist DB data?*

Using Docker volumes or bind mounts

```
docker run -v /data:/var/lib/mysql mysql
```

- **/data (Source):** This is the folder on your **Host Machine** (your actual laptop or server).
- **:**: Think of this as the "link" or "mapping" symbol.
- **/var/lib/mysql (Destination):** This is the folder **inside the container** where the MySQL database is programmed to store all its actual data (tables, rows, etc.).

If Host Folder (/data) is...	Result
Empty	Container files(/var/lib/mysql) are copied to Host. (Good for first-time setup).
Not Empty	Host files "hide" the Container files(/var/lib/mysql). (Good for resuming an old database).

Types of networks:

Network Type	Scope	Main Use Case
Bridge(default)	Single Host	Standard containers talking to each other.
Host	Single Host	Performance-critical apps (removes isolation).
Overlay	Multi-Host	Docker Swarm / Microservices across servers.
Macvlan	Local Network	Making a container look like a physical machine.
None	None	Total isolation for high-security tasks.

Docker Networking

By default, containers are isolated environments. Docker Networking allows containers to talk to each other, to the host machine, and to the outside world (like the public internet).
When you install Docker, it automatically creates three default networks: **bridge**, **host**, and **none**. However, Docker provides several network drivers depending on your needs:

Core Network Drivers:

- **Bridge Network (Default):**
  - **How it works:** When you start a container without specifying a network, it connects to a default internal bridge network (usually named bridge). Docker acts as a virtual router, assigning a private IP address to each container.
  - **Use Case:** Best for isolated containers running on the *same* host machine that need to talk to each other.
  - *Note:* It is highly recommended to create **User-Defined Bridge Networks** instead of using the default one. User-defined bridges provide automatic **DNS resolution** (containers can communicate using their container names as hostnames instead of shifting IP addresses).
    - `docker network create --driver bridge my_isolated_network`

```
# Start a database container named "db"
```

- `docker run -d --name db --network my_isolated_network postgres:15`

Start a web app container named "web" on the same network

- `docker run -d --name web --network my_isolated_network -p 8080:8080 my-web-app`

- **Host Network:**
  - **How it works:** The container shares the host machine's networking namespace directly. It doesn't get its own isolated IP address; if a container runs an app on port 80, it occupies port 80 directly on the host machine.
  - **Use Case:** Maximizes network performance because there is no network address translation (NAT) overhead.
- **Overlay Network:**
  - **How it works:** Connects multiple Docker daemons across different physical or virtual host machines.
  - **Use Case:** Essential for multi-host clustering environments, such as Docker Swarm or multi-node systems, allowing containers on Server A to talk securely to containers on Server B.
- **None Network:**
  - **How it works:** Completely disables the networking stack for the container. It has no external network interface, only a loopback interface (localhost).
  - **Use Case:** Ideal for highly secure, isolated batch jobs or data processing tasks that must not touch the network.

List networks:	`docker network ls`

Docker Storage (Data Persistence)

By default, containers are **ephemeral (temporary)**. Any data created or modified inside a container is stored in a writable container layer. When the container is deleted, **that data is permanently lost**.

To save data permanently or share it between containers, Docker provides two primary mechanisms: **Volumes** and **Bind Mounts**.

A. Volumes (Recommended)

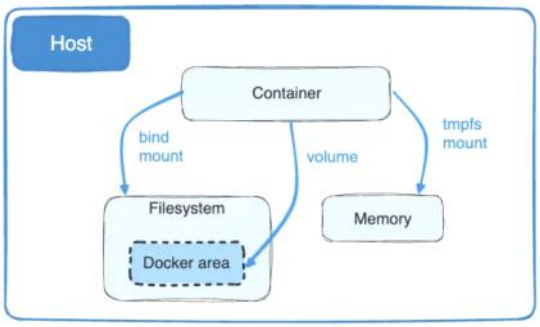
- **What they are:** Storage directories managed entirely by Docker on the host machine's filesystem (usually under /var/lib/docker/volumes/ on Linux).
- **Characteristics:**
  - Isolated from the host machine's direct OS operations; non-Docker processes shouldn't modify this directory.
  - Easy to back up, migrate, or share among multiple running containers.
  - Supports volume drivers, allowing you to store data on cloud providers or remote storage arrays.
- **Use Case:** Standard databases (like PostgreSQL or MySQL) where data safety, performance, and volume management are critical.

B. Bind Mounts

- **What they are:** A direct link to *any* specific, user-defined path on the **host machine** (e.g., mounting /home/user/my-web-app into /app inside the container).
- **Characteristics:**
  - Highly dependent on the directory structure of the host OS.
  - Any process on the host machine can modify the files, and changes instantly reflect inside the container (and vice versa).
- **Use Case:** Local development. You can mount your source code directory into a container; as you edit code on your host laptop, the container instantly sees the changes without needing a rebuild.

C. tmpfs Mounts

- **What they are:** Storage created purely in the host machine's memory (**RAM**).
- **Use Case:** Storing sensitive state files or temporary credentials that should never be written to disk or the container layer for performance or security reasons.



Interview Question

How containers communicate?

- Same network → use container name
- Different → need port exposure

Docker Compose

While Docker commands work great for single containers, managing a multi-container application (e.g., a frontend web app, a backend API, a Redis cache, and a PostgreSQL database) via the CLI becomes unmanageable. You would have to manually run multiple docker run commands, remember all network attachments, and manage port exposures.
Docker Compose is a tool used to define and run multi-container Docker applications using a single configuration file written in YAML format (typically named compose.yaml or docker-compose.yml).

- **Docker Compose:** Handling Multiple containers ---->>> On **Single** server/host.
- **Docker Stack:** Handling Multiple containers ---->>> On **Multiple** servers/hosts (a Swarm cluster).

Key Features of Docker Compose:

1. **Single-Command Management:** You can build, create, start, and attach networks/volumes to all services using just one command: docker compose up. You can stop and clean them up with docker compose down.
2. **Isolated Environments:** Compose creates a dedicated network for your application by default. Every container defined in that file can instantly communicate with others using their service name as the hostname.
3. **Preservation of Volume Data:** Compose does not destroy your volumes when you recreate containers, ensuring database changes aren't lost during updates.
   Example of a simple compose.yaml:

When we use builtin image.

```
services:
  web-app:
    image: node:18
```

```
    ports:
      - "3000:3000"
    volumes:
      - ./src:/app/src
    networks:
      - backend-network
    depends_on:
      - database
database:
    image: postgres:15
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secretpassword
    volumes:
      - db-data:/var/lib/postgresql/data
    networks:
      - backend-network
volumes:
  db-data:
networks:
  backend-network:
```

When we need to use custom image via docker file:

```
services:
  web-app:
    build: .   # 👈 Tells Compose: "Look in this current directory for
a Dockerfile and build it!"
    ports:
      - "3000:3000"
```

Docker Swarm?

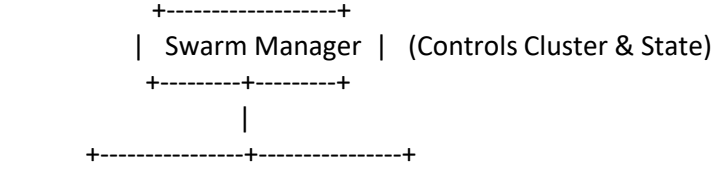
Docker Swarm is Docker's native, built-in **container orchestration platform**. It turns a group of separate physical or virtual machines running Docker into a single, virtual, highly available router and computing cluster.

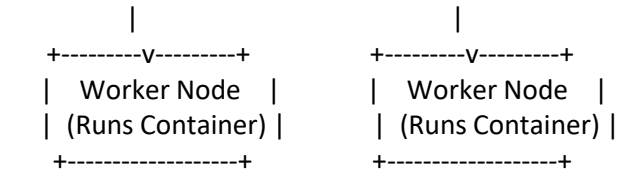
If you are familiar with **Kubernetes**, Docker Swarm serves the exact same primary purpose but is significantly simpler to configure, manage, and operate because it is integrated directly into the standard Docker CLI.

Docker Stack: is the command suite you use to deploy your application blueprints, **Docker Swarm** is the actual container orchestration tool that manages the cluster of machines, handles scheduling, ensures high availability, and maintains load balancing.

Swarm Architecture: Managers vs. Workers

A Docker Swarm cluster is made up of two types of nodes (servers):





A. Manager Nodes

- **The Brains:** Manager nodes handle cluster management tasks, maintain the cluster state, schedule services, and serve the Swarm HTTP API endpoints.
- **Raft Consensus:** If you have multiple manager nodes (for high availability), they use the Raft Consensus Algorithm to ensure they all agree on the state of the cluster. If the primary manager dies, a new manager is elected automatically.

  The mathematical formula for a majority quorum in Raft is:

  Quorum = [N/2]+1

- Your total cluster size (N) = **2**
- [2/2] + 1 = 1 + 1 = 2
- This means that to make *any* cluster-wide decisions (like electing a leader), **2 managers must say "Yes"**.
- When one manager dies, only 1 manager is left alive. Since 1 is less than the required quorum of 2, the remaining node is legally paralyzed. It can vote for itself, but it only gets 1 vote out of the required 2.

B. Worker Nodes

- **The Muscle:** Worker nodes exist solely to execute containers (Tasks) assigned to them by the Manager nodes.
- By default, Manager nodes are also capable of running tasks just like Worker nodes, though you can configure them to be "manager-only" to save resources in production.

Core Features of Docker Swarm

Declarative Service Model & Self-Healing

Instead of spinning up individual containers, you define a **Service** (e.g., *"I want 3 replicas of my Nginx web server running"*). Swarm constantly monitors the cluster state. If a worker node crashes or a container dies, Swarm automatically detects the failure and launches replacement containers on the remaining healthy nodes to maintain your target count.

The Routing Mesh (Inbound Load Balancing)

The Swarm **Routing Mesh** ensures that any port you expose on the Swarm (e.g., port 80) is exposed on *every single node* in the cluster, even if a specific node isn't actually running a copy of that container. When traffic hits any node, the routing mesh automatically forwards the request internally to a node that is running the active container.

Secure by Default

All communication between nodes in a Docker Swarm is automatically secured out-of-the-box using mutual TLS (mTLS) encryption and authentication. Swarm handles certificate rotation automatically without you needing to configure anything manually.

Essential Docker Swarm Commands

To start using Swarm, you don't need to install anything extra; you just change Docker from its standard single-host mode into Swarm mode.

Command	Description	Where to

		Run It
`docker swarm init --advertise-addr <MANAGER-IP>`	**Initializes a new Swarm cluster** and sets the current machine as the primary Manager node. Outputs a join token.	Primary Manager Server
`docker swarm join --token <TOKEN> <MANAGER-IP>:2377`	**Joins a separate server** into your existing Swarm cluster as a Worker node.	New Worker Server
`docker node ls`	**Lists all nodes** currently attached to the cluster, along with their status, availability, and roles.	Any Manager Node
`docker stack services my-company-web`	**Lists active services** within a deployed stack, showing the targeted vs. running replica count.	Any Manager Node
`docker stack ps my-company-web`	**Shows the exact placement** of every single container replica across the physical nodes in the cluster.	Any Manager Node
`docker stack rm my-company-web`	**Tears down the stack**, completely deleting all associated containers, services, and overlay networks.	Any Manager Node
`docker swarm leave --force`	**Forces the current server to leave** the Swarm cluster, disabling Swarm mode on that specific machine.	Target Node

Core Comparison Table

Feature / Criteria	Docker Swarm	Kubernetes (K8s)
Installation & Setup	**Extremely Simple.** Built into the Docker Engine (docker swarm init). Up and running in seconds.	**Complex.** Requires separate tools (like kubeadm, Minikube, or cloud providers) and steep initial configuration.
Learning Curve	**Low.** Uses the familiar Docker CLI and standard Compose YAML syntax.	**High.** Requires learning an entirely new ecosystem of concepts (Pods, Deployments, CRDs, etc.).
Atomic Unit	**Container/Task.** Runs individual containers directly mapped to services.	**Pod.** A wrapper around one or more tightly coupled containers that share network/storage.
Scaling & Speed	Fast container deployment. Less overhead makes scaling up/down near-instant.	Slightly slower deployment due to extensive background validation and state checking, but highly reliable.
Auto-Scaling	**Manual/Basic.** Requires external scripts or third-party tools to automatically scale based on traffic.	**Built-in.** Automatically scales pods up or down (Horizontal Pod Autoscaler) and adds cluster nodes natively.
Load Balancing	Built-in via the **Routing Mesh** (Ingress routing across all cluster nodes out-of-the-box).	Requires manual setup or cloud provider integrations (Ingress controllers, LoadBalancer services).
Customization	**Limited.** You take what Docker gives you. Hard to plug in custom networking or storage layers.	**Infinite.** Highly modular. You can swap out the networking (Calico, Flannel), storage, or runtime engines easily.

Why Choose Docker Swarm?

- **Local Development & Small Teams:** If you have a small team or want to quickly deploy a

multi-node app without managing complex infrastructure.

- **Low Overhead:** It uses fewer system resources (RAM/CPU) than Kubernetes just to keep the orchestration management layer alive.
- **Familiarity:** If you already know Docker Compose, you already 90% know Docker Swarm.

Why Choose Kubernetes?

- **Massive Scale:** It is the industry standard for enterprise-level applications managing thousands of containers.
- **High Complexity & Microservices:** If your app requires intricate traffic routing, complex storage management, or automated scaling based on advanced custom metrics.
- **Cloud Native Ecosystem:** It integrates deeply with every major cloud provider (AWS EKS, Google GKE, Azure AKS) and features a massive community marketplace of tools (Helm charts, Prometheus, Istio).

Interview Question

☞ *Your image size increasing, what do you do?*

Answer:

- Multi-stage builds
- Minimal base image
- Remove unnecessary files
- Optimize layers

Multi stage Builds:

Analogy:

To make a cake, you need a messy counter with flour, eggshells, a heavy mixer, and bowls. But when you serve the cake to the customer, you don't give them the eggshells and the mixer—you just give them the finished cake on a clean plate.

In Docker, a multi-stage build does exactly this: it uses a "messy" environment to build your app and then moves only the finished "product" into a tiny, clean container.

Why do we use it?

Without multi-stage builds, your Docker image includes everything: the source code, the compiler, the build tools, and the heavy libraries. This makes the image **huge** (e.g., 800MB).

With multi-stage builds, you can get that same image down to **20MB** because it only contains the executable file.

How it works (The Dockerfile)

- You use the FROM command multiple times in a single Dockerfile. Each FROM starts a new "stage."

- **1. The "State" Reset:** When Docker sees a **new FROM** instruction, it tells the builder: *"Okay, stop everything you are doing. Put the current container aside, and start a brand new, empty container using this new image."*
- **2. The "Bridge" (COPY --from)** If the rooms are locked and separate, how do you get your app into the new room? That is what COPY --from=builder does.
  It is like a **small window** between the two rooms. You reach back into the "Golang Room," grab only the finished executable, and pull it into the "Alpine Room."

The Key Differences

Feature	Single Stage	Multi-Stage
Image Size	Massive (includes compilers/tools).	Tiny (includes only the app).
Security	Lower (attackers can use build tools).	Higher (no tools available for hackers).
Simplicity	One big block.	Multiple "FROM" blocks.
Analogy	Giving a customer the cake AND the oven.	Giving the customer just the cake.

1. Without Multi-Stage Build

This version is "the messy kitchen." It keeps the compiler, the source code, and all the build tools inside the final image that you deploy.

Dockerfile:

```
# Use the heavy library that includes the Go compiler
FROM golang:1.21
# Set the working directory
WORKDIR /app
# Copy the source code into the image
COPY . .
# Compile the application
RUN go build -o my-app
# The command to run the app
CMD ["./my-app"]
```

- **Resulting Image Size:** ~800MB to 1GB.
- **What's inside:** The OS, Go compiler, source code, build cache, and the executable.

2. With Multi-Stage Build

This version is "the clean plate." We use a heavy image to build the app, then move the result to a tiny, "distroless" or lightweight image.

Dockerfile:

Dockerfile

```
# STAGE 1: The Builder (The Kitchen)
FROM golang:1.21 AS builder
WORKDIR /app
COPY . .

# Build the static binary
RUN go build -o my-app

# STAGE 2: The Final Image (The Plate)
FROM alpine:latest
WORKDIR /root/

# Copy ONLY the compiled binary from the 'builder' stage
COPY --from=builder /app/my-app .

# Run the tiny binary
```

```
CMD ["./my-app"]
```

- **Resulting Image Size:** ~15MB to 20MB.
- **What's inside:** Just the tiny Alpine OS and your single executable file. No source code, no compiler.

Environment Config

"Build Once, Run Anywhere." The goal is to make sure the exact same "package" (the Docker image) works in Dev, Staging, and Production without you having to change the code or the Dockerfile.

Problem:

Single Dockerfile, multiple environments. If we have multiple docker files, Configuration drift may cause - features will get deviated.

Configuration Drift happens when your Dev environment and Prod environment start looking different because humans manually changed files in one but forgot the other.

Solution:

Use ENV variables

```
ENV APP_ENV=dev
```

Pass at runtime:

```
docker run -e APP_ENV=prod app
```

Use .env file

```
docker run --env-file .env app
```

Analogy:

The Simple Analogy: The "Universal Remote Control"

Imagine you are manufacturing a **Universal Remote Control**.

- **The Wrong Way (Multiple Dockerfiles):** You build a special remote for Sony TVs, a different one for Samsung, and another for LG. If you want to change the color of the buttons, you have to go to three different factories and update three different blueprints. This is **Configuration Drift** (eventually, the Sony remote will have features the LG one doesn't).
- **The Right Way (Single Dockerfile):** You build **one** remote with a "Sync" button. When the user takes it home, they point it at their TV (the environment) and it "learns" how to behave. The remote is the same; the **settings** it receives at the destination are different.

How it works in Docker (Simple Terms)

1. The Dockerfile (The Blueprint)

You write your Dockerfile to look for **Environment Variables** instead of hardcoded values.

In your code, instead of saying connect to dev-db.com, you say connect to $DATABASE_URL.

2. The Injection (The "Sync" Button)
When you start the container, you "inject" the specific settings for that environment.

- **For Development:** docker run -e DB_URL=dev-db.com -e API_KEY=abc1234 my-app
- **For Production:** docker run -e DB_URL=prod-db.com -e API_KEY=xyz9876 my-app

The **Image** is identical. Only the **Values** changed at the moment of starting.

Which values should use environmental variables:

1. Secrets and Credentials (The Most Important)
Never commit passwords or API keys to git.

- **Examples:** DATABASE_PASSWORD, STRIPE_API_KEY, JWT_SECRET, AWS_SECRET_ACCESS_KEY.

2. Service Endpoints and Dynamic URLs
Your app needs to point to different places depending on where it's running.

- **Examples:** DATABASE_URL (points to localhost in dev, but an AWS RDS endpoint in prod), ANALYTICS_API_URL.

3. Feature Toggles and Feature Flags
If you want to test a feature in Dev but keep it hidden in Production without rewriting code.

- **Examples:** ENABLE_NEW_DASHBOARD=true, MAINTENANCE_MODE=false.

4. Operational Settings
Variables that change how the application runs or scales on a specific server.

- **Examples:** LOG_LEVEL=debug (verbose logs for dev) vs LOG_LEVEL=error (clean logs for prod), PORT=3000.

Secrets (IMPORTANT)

- Never hardcode
- Use:
    - Vault
    - Kubernetes secrets (later)

Managing Credentials (The "Safe" Analogy)

You wouldn't tape your house key to the front of the Remote Control (Hardcoding). If you did, everyone who sees the remote knows your key.
Instead, you give the Remote a **Code** to open a **Safe** that is already at the house.

1. **Vault / Secrets Manager:** In Production, your container doesn't "know" the password. It reaches out to a secure service (like HashiCorp Vault or AWS Secrets Manager) and says: "I am the Production App, give me my database password."
2. **Kubernetes Secrets:** If using K8s, you store the password in a "Secret" object. When the container starts, Kubernetes "mounts" that password into the container as a file or a variable.

Why this prevents "Configuration Drift"?

Configuration Drift happens when your Dev environment and Prod environment start looking different because humans manually changed files in one but forgot the other.
By using a **Single Dockerfile**:

- If the app works in Dev, it **will** work in Prod because the binary, the OS version, and the dependencies are **100% identical**.
- The only thing that changes is the "Address Book" (Environment Variables) you give it.

Interview Answer

> “I use environment variables and external configuration instead of creating multiple Dockerfiles, ensuring consistency and avoiding configuration drift.”

The goal: The application will greet a user. The "Greeting" and the "Password" (Secret) will change depending on the environment, but the **Docker image will stay exactly the same.**

1. The Application Code (app.py)

This script doesn't have "Dev" or "Prod" written inside it. It just looks for environment variables.
Python

```
import os
# Get configuration from Environment Variables
# The second argument is a 'default' value if the variable isn't found
env_name = os.getenv("APP_ENV", "development")
greeting = os.getenv("GREETING", "Hello from Local")
db_password = os.getenv("DB_PASSWORD", "no-password-found")
print(f"--- Running in {env_name.upper()} mode ---")
print(f"Message: {greeting}")
print(f"Secret Password: {db_password}")
```

2. The Single Dockerfile

We use a **Multi-Stage Build** (as we discussed before) to keep it optimized. Notice there is **no mention** of "Dev" or "Prod" settings here.
Dockerfile

```
# STAGE 1: Build
FROM python:3.9-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user -r requirements.txt
# STAGE 2: Run
FROM python:3.9-slim
WORKDIR /app
# Copy only the installed packages and the code
COPY --from=builder /root/.local /root/.local
COPY app.py .
# Ensure scripts in .local/bin are usable
ENV PATH=/root/.local/bin:$PATH
CMD ["python", "app.py"]
```

3. The Commands

Step A: Build the Image (One time)

You build this once. This image is now your "Gold Standard."
Bash

```
docker build -t universal-app .
```

Step B: Run in "Development"

You pass the dev-specific variables using the -e flag.

```
docker run --name app-dev \
```

```
  -e APP_ENV=development \
  -e GREETING="Hey Dev! Keep coding!" \
  -e DB_PASSWORD="dev_pass_123" \
  universal-app
```

Step C: Run in "Production"
You use the **exact same image** but pass different, secure variables.

```
docker run --name app-prod \
  -e APP_ENV=production \
  -e GREETING="Welcome to the Production System" \
  -e DB_PASSWORD="SUPER_SECRET_PROD_PASSWORD_99" \
  universal-app
```

4. Using an Environment File (.env)

For a real project, you don't want to type 20 variables in the command line. You use a .env file.
Create a file named prod.env:

```
APP_ENV=production
GREETING=Welcome to the Production System
DB_PASSWORD=SUPER_SECRET_PROD_PASSWORD_99
```

Run it using the file:

```
docker run --env-file prod.env universal-app
```

Final Interview Tip:

If they ask: *"Where do you store that prod.env file?"*

- **Answer:** "I don't store it in Git. I store it in a secure location like **AWS Secrets Manager**, **HashiCorp Vault**, or as a **Kubernetes Secret**. During the CI/CD pipeline, the secret is pulled and injected into the container."

Docker Compose vs. Docker Swarm

- **Docker Compose:** Used for **defining and running** multi-container apps on a **single host** (e.g., your laptop). It uses the docker-compose.yml file.
- **Docker Swarm:** A **clustering/orchestration** tool. It turns multiple Docker hosts into a single virtual host. (Note: Mentioning **Kubernetes** as the more popular alternative to Swarm will score you bonus points)

Namespaces and Cgroups (The "Magic" under the hood)

If they ask, "How does Docker actually isolate a container?" they are looking for these two Linux Kernel features:

- **Namespaces:** Provide **Isolation**. (It makes the container think it has its own private network, users, and files).
- **Cgroups (Control Groups):** Provide **Resource Limits**. (It prevents one container from eating all the CPU or RAM of the host).

Security Best Practices

If image size is the #1 optimization, **Security** is #2.

- **Run as Non-Root:** By default, containers run as root. You should use the USER command in your

Dockerfile to switch to a non-privileged user.

- **Scan for Vulnerabilities:** Mention tools like **Trivy** or docker scan to find security holes in your base images.
- **No Secrets in Images:** Never use ENV for passwords. Use **Secrets Management** (as we discussed with Vault).

Lifecycle of Docker containers.

A Docker container passes through a lifecycle that defines the states a container can be in and how it operates in those states. The stages in a Docker container lifecycle are:

- **Create:** In this state, we set up a container from an image with the `docker create` command.
- **Run**: Here, we use the `docker start` command to run the container, which performs the tasks until we stop or pause it.
- **Pause**: We use the `docker pause` command to stop the process. This state keeps the memory and disk intact. If you want to resume the container, use the `docker unpause` command.
- **Stop**: If the container is inactive, it enters the stop stage, but this can happen due to multiple reasons:
  - **Immediate stop**: The `docker kill` command stops the container without cleanup.
  - **Process completion**: When the container finishes the task, it stops automatically.
  - **Out of memory**: Container stops when it consumes too much memory.
- **Delete**: In the final stage, we remove the stopped or created container with the `docker rm` command.

Signing Docker Images

[1][2][3]

Signing Docker images ensures their **integrity** and authenticity. **Docker Content Trust** (DCT) provides the ability to use **digital signatures** for data sent to and received from remote Docker registries.

Docker signature tools: Cosign, Docker Content Trust

Via Cosign:

Signing Docker container images ensures authenticity and prevents unauthorized or tampered images from running in your environments.
Today, the modern industry standard for signing container images is **Cosign** (part of the Linux Foundation's Sigstore project), which has largely replaced the legacy Docker Content Trust (Notary v1).

Step 1: Generate a Key Pair

First, generate a cryptographic key pair using Cosign to sign your images.

1. **Install Cosign** (if not already installed):
   Bash

   # Linux x86_64 installation
   curl -O -L "
   https://github.com/sigstore/cosign/releases/latest/download/cosign-linux-amd64"
   sudo mv cosign-linux-amd64 /usr/local/bin/cosign
   sudo chmod +x /usr/local/bin/cosign
2. **Generate Key Pair**:
   Bash

   cosign generate-key-pair
   - **You will be prompted to enter a password to encrypt your private key (cosign.key).**
   - ***Verification Step:*** **Confirm that cosign.key (private key) and cosign.pub (public key) are created in your current working directory.**

Step 2: Build, Push, and Sign the Docker Image

Unlike raw file archives, image signatures in OCI/Docker registries are attached **after** the image (or manifest) is published to the registry. The signature itself is stored inside the registry alongside the image tag.

1. **Build and Push Your Image**:
   Bash

   # Build the image
   ```
   docker build -t gcr.io/your-project-id/my-app:v1.0.0 .
   ```
   . (dot) represents, check the dockerfile from the current folder.

2. # Push the image to your container registry (GCR, Artifact Registry, Docker Hub, etc.)
   ```
   docker push gcr.io/your-project-id/my-app:v1.0.0
   ```
3. **Sign the Pushed Image**:
   Sign the remote registry image using your private key:
   Bash

   ```
   cosign sign --key cosign.key gcr.io/your-project-id/my-app:v1.0.0
   ```

Verification Step: Cosign will log into the registry and upload a signature payload object attached to your image's digest.

Step 3: Verify the Signed Image

Provide your public key (cosign.pub) to your customers or deployment systems so they can verify the image before running it.
1.Verify Image via Cosign CLI:1 min.

Run the verification command using the public key against the image tag or digest:
Bash

```
cosign verify --key cosign.pub gcr.io/your-project-id/my-
app:v1.0.0
```

Verification: You will receive an output containing Verification for gcr.io/... -- successful! along with the signature payload claims.

2.Enforce Verification in Kubernetes (Optional):2 min.
In a production Kubernetes cluster, you can enforce that **only signed images** can be deployed by using policy engines like **Kyverno** or **OPA Gatekeeper**.

Example Kyverno ClusterPolicy snippet:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-image-signature
spec:
  validationFailureAction: Enforce
  rules:
    - name: verify-signature
      match:
        any:
        - resources:
            kinds:
              - Pod
      verifyImages:
        - imageReferences:
            - "gcr.io/your-project-id/*"
          key: |
            -----BEGIN PUBLIC KEY-----
            <contents-of-cosign.pub>
            -----END PUBLIC KEY-----
```

Verification: Try running an unsigned image in the cluster; Kubernetes will reject the creation request with an admission webhook error.

Via Docker Content Trust:

To sign a Docker image, follow these steps:

- **Generate a Key Pair**:
  `docker trust key generate your-name`

- **Add Your Public Key to Your Registry**:

```
docker trust signer add --key cert.pem your-key-name
registry.example.com/my-image
```

- **Sign the Image**:

```
docker trust sign registry.example.com/my-image:latest
```

Important Considerations

Verifying Trusted Images

- To verify the integrity of an image, set the DOCKER_CONTENT_TRUST environment variable:

```
export DOCKER_CONTENT_TRUST=1
docker pull registry.example.com/my-image:latest
```

- This ensures that only signed images are pulled.

Revoking Trust

- If you need to revoke trust for an image, use:

```
docker trust revoke registry.example.com/my-
image:latest
```

- This will delete the image's trust data.

Alternative Solutions

- For automated workflows, you can set the DOCKER_CONTENT_TRUST environment variable and use docker push:

```
export DOCKER_CONTENT_TRUST=1
docker push registry.example.com/my-image:latest
```

- This will automatically sign and push the image.

- By signing Docker images, you enhance security and ensure that users can trust the images they pull from your registry.

Interview Questions:

https://www.datacamp.com/blog/docker-interview-questions